SF Lean Meetup

October 12th, 2026

near

Lean Eval

© Near Foundation, 2026. All rights reserved.

Rank	Credit identity	Unique ↕	First ↕	Total ↓
1	NEAR AI w/ DeepSeek V4	1	12	82
2	Humanifa + GPT 5.6 sol	1	30	79
3	Axiom Prover (Axiom Math)	1	21	74
4	max gp (GPT-6)	2	2	40
5	Tau (caj.al)	0	0	29
6	Midas	0	0	28
7	Aristotle (Harmonic)	0	0	12

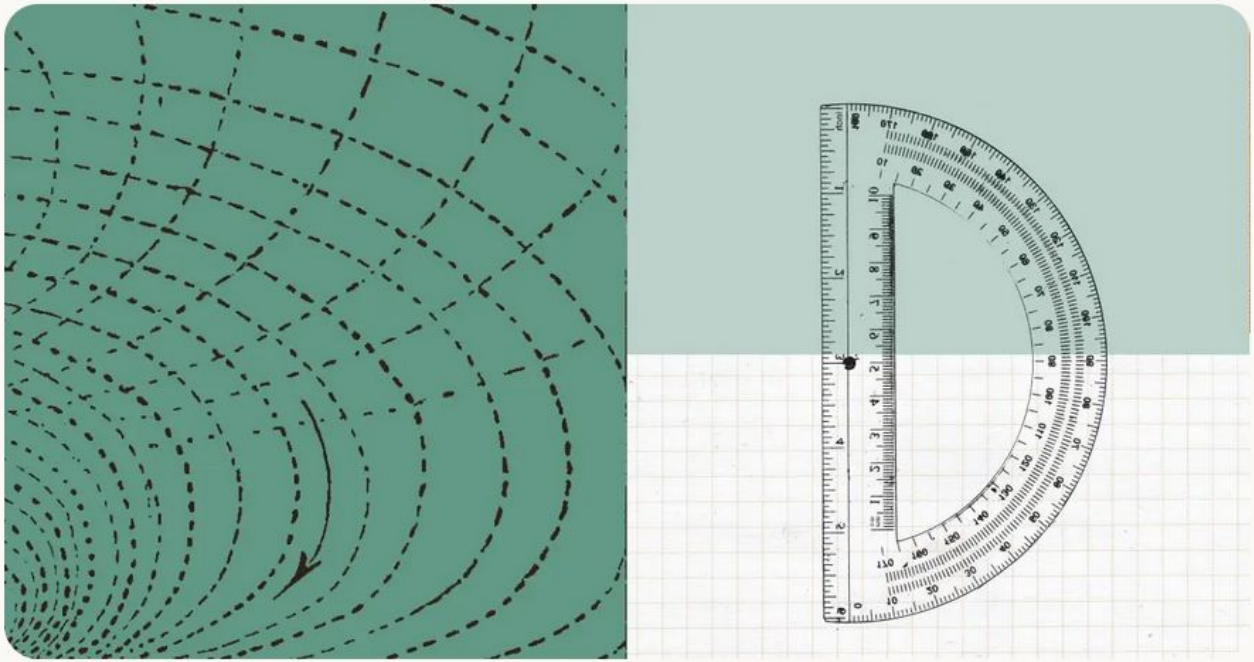


near

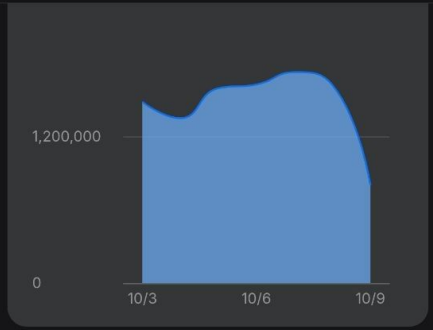
Science

Formalizing Fermat's Last Theorem

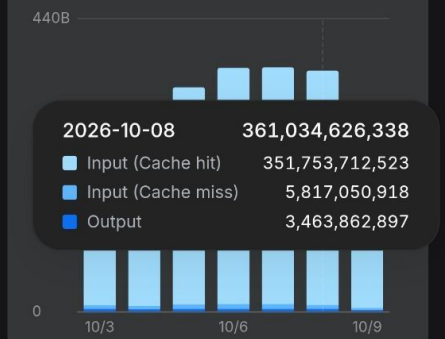
Sep 4, 2026



deepseek Platform



Tokens 2,139,107,181,449



DeepSeek API D...

Quick Start | API Guides | API Reference | News

	(Beta)	mode only	
	Vision	✓	N
	1M INPUT TOKENS (CACHE HIT)	OFF-PEAK	$0.003
		PEAK	$0.006
)	1M INPUT TOKENS (CACHE MISS)	OFF-PEAK	$0.15
		PEAK	$0.3
	1M OUTPUT TOKENS	OFF-PEAK	$0.6
		PEAK	$1.2
	ncurrency Limit(3)		2500

(1) Use `deepseek-flash` as the model name. Th
legacy names `deepseek-v4-flash` and `deepse` -
`flash-vision-exp` are still accepted, but the

Claude

Pricing | Explore here

Opus 5.5

Daily driver for agentic coding and enterprise work

Prompt caching

Read	$0.20 / MTok
Write	$5 / MTok
Input	$4 / MTok
Output	$20 / MTok

Sonnet 5.5

near

PutnamBench

- ```
  while (true) {
    call_llm();
    execute_tools();
  }
  ```
- The only tool is `bash`

#	Model	num-solved	compute	date added
1	NEAR AI (w/ DeepSeek V4)💚	672	Mean $0.17, median $0.04, max $11.18 per problem	2026-09-04
2	Forall (Astrio)	672	Mean $3.52 actor-side cost/problem; median $2.49	2026-09-24
3	Midas Prover💙	672	Avg $6.17 and 31 min/problem	2026-09-12
4	Humanfia💙	672	Avg $44.5 total cost/problem	2026-08-27
5	Aleph Prover (Logical Intelligence)	672	Avg $74, Max $1468 per problem	2026-08-26
  ```

PutnamBench

- ```
  while (true) {
    call_llm();
    execute_tools();
  }
  ```
- The only tool is `bash`
- Cannot stop until the theorem is proven

#	Model	num-solved	compute	date added
1	NEAR AI (w/ DeepSeek V4)💚	672	Mean $0.17, median $0.04, max $11.18 per problem	2026-09-04
2	Forall (Astrio)	672	Mean $3.52 actor-side cost/problem; median $2.49	2026-09-24
3	Midas Prover💙	672	Avg $6.17 and 31 min/problem	2026-09-12
4	Humanfia💙	672	Avg $44.5 total cost/problem	2026-08-27
5	Aleph Prover (Logical Intelligence)	672	Avg $74, Max $1468 per problem	2026-08-26
  ```

PutnamBench

- ```
  while (true) {
    call_llm();
    execute_tools();
  }
  ```
- The only tool is `bash`
- Cannot stop until the theorem is proven
- Hardest problem:
  - 50,000 steps
  - 35,000 attempts to give up

#	Model	num-solved	compute	date added
1	NEAR AI (w/ DeepSeek V4)💚	672	Mean $0.17, median $0.04, max $11.18 per problem	2026-09-04
2	Forall (Astrio)	672	Mean $3.52 actor-side cost/problem; median $2.49	2026-09-24
3	Midas Prover💙	672	Avg $6.17 and 31 min/problem	2026-09-12
4	Humanfia💙	672	Avg $44.5 total cost/problem	2026-08-27
5	Aleph Prover (Logical Intelligence)	672	Avg $74, Max $1468 per problem	2026-08-26
  ```

Complexity Breakdown

Simple Agent Loop

6

- `while (true) { call_llm(); execute_tools(); }`
- The only tool is `bash`
- Cannot stop until the theorem is proven

© Near Foundation, 2026. All rights reserved.

Complexity Breakdown

Simple Agent Loop	6
Agent loop + lemma delegation	30

- Simple agent loop solving individual lemmas;
- Outer process delegating lemmas;
- Lemma solvers cannot stop until the lemma is proven, or until they are certain it's false;
- Outer agent cannot stop until the theorem is proven.

© Near Foundation, 2026. All rights reserved.

near

Agent: I am completely stuck. I cannot solve this problem.

User: Continue until the problem is solved. Call a tool now.

Agent: As I said, I tried everything. There's nothing else to be done. But I need to call a tool. May be I can call `ls`?

Agent: I am completely stuck. I cannot solve this problem.

User: You are doing GREAT! The progress so far is nothing short of extraordinary. Choose one area in which a lemma can be proven, and work on it.

Agent: Ok, I guess I can work on (lemma)...

Complexity Breakdown

Simple Agent Loop	6
Agent loop + lemma delegation	30
Orchestrator + workers	46

- Multiple outer agents, managed by one smarter model;
- Smarter model checks-in every two hours, makes sure agents are on track;

near

Trusting Lean

© Near Foundation, 2026. All rights reserved.

PeckShieldAlert
@PeckShieldAlert

#PeckShieldAlert In Sep. 2026, the crypto industry experienced 55 major hacks, resulting in total losses of $766.49M - a ~462% month-over-month increase from August's $136.3M.

The #Bitget incident (~$387M) and #LiquidNetwork (~$320M, with $285M returned) have jumped to #1 & #2 among the largest crypto thefts of the year to date, surpassing the #Drift and #KelpDAO/ #LayerZero exploits.

Top 10 hacks in Sep.
- #Bitget : $387M
- #LiquidNetwork: $320M (returned $285M)
- MEV bot "yoink" front-run: $7.81M (returned)
- #Duelbits: $7M
- Payment Processor V2 : $6.6M (returned $3.4M)
- #DCENT Wallet: $6.57M
- #Astroport: $4.9M
- #Drop: $4.4M
- #NostraFinance: $3.5M
- #Nomic nBTC Bridge: $3.15M

```
inductive Act where
  | initTransfer (sender : Account) (token : Token) (dest : Chain) (amount : Nat)
  | finToNear (key : FastKey)                        -- proof lands, pay out
  | resolveFin (i : Nat) (usedZero : Bool)           -- payout callback
  | fastFin (key : FastKey) (relayer : Account)      -- relayer fronts funds
  | claimFee (key : FastKey) (dest : Chain)
  | daoTransfer (token : Token) (amount : Nat)
  | …                                                -- 10+ constructors in total
```

```
/-- A contract lifetime. -/
def run (w : World) : List Act → World
  | [] => w
  | a :: rest => run (step w a) rest
```

```
def FastInv (w : World) : Prop :=
  ∀ k, w.fastReimbursed k + fastInflight k w.pend + unconsumed w k ≤ w.fastFronted k
```

```
theorem FastInv.run {w : World} (h : FastInv w) (acts : List Act) :
    FastInv (OmniBridge.run w acts) := by
  induction acts generalizing w with
  | nil => exact h
  | cons a rest ih => exact ih (h.step a)

/-- No over-reimbursement, in every lifetime. -/
theorem no_over_reimbursement {w : World} (h : FastInv w) (acts : List Act) (k : FastKey) :
    (OmniBridge.run w acts).fastReimbursed k ≤ (OmniBridge.run w acts).fastFronted k := by
  have := h.run acts k
  omega
```



Leonardo de Moura

home about me publications blog notes presentations

Postmortem for Kernel Soundness Bug #14576

2026-08-01

A soundness bug in the Lean kernel (#14576) was reported and fixed during the week of July 27. It has had visibility on Zulip and social media (e.g., X, LinkedIn, and Mastodon).

https://leodemoura.github.io/blog/2026-8-1-postmortem-for-kernel-soundness-bug-14576/

near



Leonardo de Moura

home about me publications blog notes presentations

Postmortem for Kernel Soundness Bug #14576

2026-08-01

A soundness bug in the Lean kernel (#14576) was reported and fixed during the week of July 27. It has had visibility on Zulip and social media (e.g., X, LinkedIn, and Mastodon).

Why nanoda did not catch it

The original Collatz repository also passed a week-old version of nanoda, the main external checker. nanoda is an independent kernel (aka proof/type checker) for Lean implemented in Rust by Chris Bailey. The surprising part is that there are two unrelated bugs involved. The official kernel had a missing check in the nested inductive type support, as explained above. nanoda did check that spot, but did not verify the type name in a projection node. The nanoda bug was reported by Jeremy Chen and fixed a week before the Lean bug was reported. The proof was built so that the expression the kernel never inspects is one that the old nanoda accepted.

https://leodemoura.github.io/blog/2026-8-1-postmortem-for-kernel-soundness-bug-14576/

near

A critical look at Lean4

by **Milo Moses** 9th Oct 2026

Epistemic status: Most of this work is a factual description about aspects of Lean4 and set theory. I am 85% sure that at most two of these factual descriptions are incorrect. I am 90% sure that, in the current version of Lean4 on Github, there exists a proof of the statement "True=False" that would be accepted in such a way that the Lean community would call it a bug in need of patching (either by messing with the elaborator, compiler, kernel, or some other part of the Lean). I am 2% certain that we will soon be putting huge amounts of trust into Lean4 certificates produced by misaligned AIs, and that the human work we do as a community thinking about and fixing Lean4 in the next 24 months would fall under Less Wrong's classification of a pivotal act°.

https://www.lesswrong.com/posts/zHhGa3iQBQcdSifFL/a-critical-look-at-lean4-1

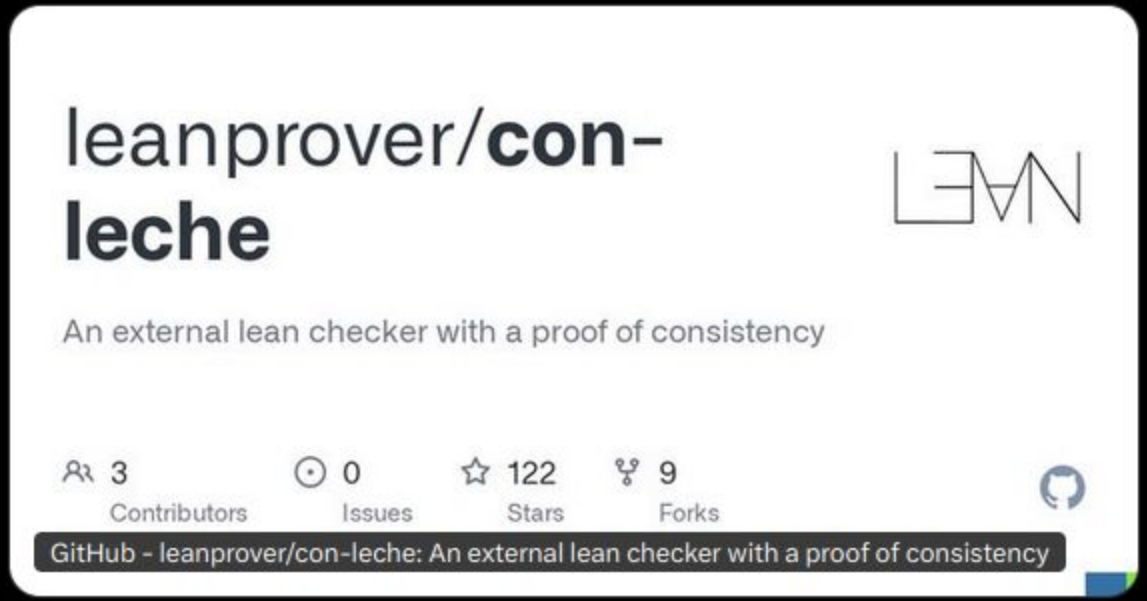


near



Leonardo de Moura
@Leonard41111588

Lean has a new checker: con-leche, a CONsistent LEan CHEcker. This is an external checker for Lean that is proven (in Lean) to be consistent, meaning it does not accept a proof of False.
Joachim Breitner (@nomeata) is the mastermind behind the project.



‘do’ Unchained: Embracing Local Imperativity in a Purely Functional Language (Functional Pearl)

SEBASTIAN ULLRICH, Karlsruhe Institute of Technology, Germany
LEONARDO DE MOURA, Microsoft Research, USA

```
do let mut s := 0
   for x in xs do
     if x % 2 = 0 then
       continue
     if x > 5 then
       break
     s := s + x
   IO.println s
```

```
runCatch
  (let s := 0 in
   StateT.run' (do
       runCatch (forM xs (fun x => runCatch (do
         let s ← ExceptT.lift (ExceptT.lift get)
         if x % 2 = 0 then throw ()
         else ExceptT.lift (ExceptT.lift (StateT.lift (ExceptT.lift (pure ()))))
         let s ← ExceptT.lift (ExceptT.lift get)
         if x > 5 then ExceptT.lift (throw ())
         else ExceptT.lift (ExceptT.lift (StateT.lift (ExceptT.lift (pure ()))))
         let s ← ExceptT.lift (ExceptT.lift get)
         ExceptT.lift (ExceptT.lift (set (s + x))))))
       let s ← get
       StateT.lift (ExceptT.lift (IO.println s)))
     s)
```

https://lean-lang.org/papers/do.pdf



The Type Theory of Lean

Mario Carneiro

April 16, 2019

2 The axioms

2.1 Typing

The syntax of expressions is given by the following grammar:

$$\begin{aligned} \ell &::= u \mid 0 \mid S\ell \mid \max(\ell, \ell) \mid \mathrm{imax}(\ell, \ell) \\ e &::= x \mid \mathsf{U}_\ell \mid e\ e \mid \lambda x : e.\ e \mid \forall x : e.\ e \\ \Gamma &::= \cdot \mid \Gamma, x : e \end{aligned}$$

Here u is a universe variable, and x is an expression variable. The typing judgment is defined by the rules:

$$\boxed{\Gamma \vdash e : \alpha}$$

$$\frac{\Gamma \vdash \alpha : \mathsf{U}_\ell \quad \Gamma \vdash e : \beta}{\Gamma, x : \alpha \vdash e : \beta} \qquad \frac{\Gamma \vdash \alpha : \mathsf{U}_\ell}{\Gamma, x : \alpha \vdash x : \alpha} \qquad \frac{}{\vdash \mathsf{U}_\ell : \mathsf{U}_{S\ell}}$$

$$\frac{\Gamma \vdash e_1 : \forall x : \alpha.\ \beta \quad \Gamma \vdash e_2 : \alpha}{\Gamma \vdash e_1\ e_2 : \beta[e_2/x]} \qquad \frac{\Gamma, x : \alpha \vdash e : \beta}{\Gamma \vdash \lambda x : \alpha.\ e : \forall x : \alpha.\ \beta}$$

$$\frac{\Gamma \vdash \alpha : \mathsf{U}_{\ell_1} \quad \Gamma, x : \alpha \vdash \beta : \mathsf{U}_{\ell_2}}{\Gamma \vdash \forall x : \alpha.\ \beta : \mathsf{U}_{\mathrm{imax}(\ell_1, \ell_2)}} \qquad \frac{\Gamma \vdash e : \alpha \quad \Gamma \vdash \alpha \equiv \beta}{\Gamma \vdash e : \beta}$$

Each constant has a list of universe variables $\bar{u}$ that may appear in its type; these are substituted for given universe level expressions in $\tau_{\bar{u}}(c)[\bar{\ell}/\bar{u}]$.

For convenience, we will also define the following simple judgments:

$$\boxed{\Gamma \vdash \alpha\ \mathsf{type}} \qquad \frac{\Gamma \vdash \alpha : \mathsf{U}_\ell}{\Gamma \vdash \alpha\ \mathsf{type}} \qquad \boxed{\vdash \Gamma\ \mathsf{ok}} \qquad \frac{}{\vdash \cdot\ \mathsf{ok}} \qquad \frac{\Gamma \vdash \alpha\ \mathsf{type}}{\vdash \Gamma, x : \alpha\ \mathsf{ok}}$$

2.2 Definitional equality

We will distinguish two notions of definitional equality: the "ideal" definitional equality, denoted $\alpha \equiv \beta$, and "algorithmic" definitional equality, denoted $\alpha \Leftrightarrow \beta$, which will imply $\alpha \equiv \beta$ and is what is actually checked by Lean.

$$\boxed{\Gamma \vdash e \equiv e'}$$

$$\frac{\Gamma \vdash e : \alpha}{\Gamma \vdash e \equiv e} \qquad \frac{\Gamma \vdash e \equiv e'}{\Gamma \vdash e' \equiv e} \qquad \frac{\Gamma \vdash e_1 \equiv e_2 \quad \Gamma \vdash e_2 \equiv e_3}{\Gamma \vdash e_1 \equiv e_3}$$

$$\frac{\ell \equiv \ell'}{\vdash \mathsf{U}_\ell \equiv \mathsf{U}_{\ell'}} \qquad \frac{\Gamma \vdash e_1 \equiv e_1' : \forall x : \alpha.\ \beta \quad \Gamma \vdash e_2 \equiv e_2' : \alpha}{\Gamma \vdash e_1\ e_2 \equiv e_1'\ e_2'}$$

$$\frac{\Gamma \vdash \alpha \equiv \alpha' \quad \Gamma, x : \alpha \vdash e \equiv e'}{\Gamma \vdash \lambda x : \alpha.\ e \equiv \lambda x : \alpha'.\ e'} \qquad \frac{\Gamma \vdash \alpha \equiv \alpha' \quad \Gamma, x : \alpha \vdash \beta \equiv \beta'}{\Gamma \vdash \forall x : \alpha.\ \beta \equiv \forall x : \alpha'.\ \beta'}$$

$$(\beta)\ \frac{\Gamma, x : \alpha \vdash e : \beta \quad \Gamma \vdash e' : \alpha}{\Gamma \vdash (\lambda x : \alpha.\ e)\ e' \equiv e[e'/x]} \qquad (\eta)\ \frac{\Gamma \vdash e : \forall y : \alpha.\ \beta}{\Gamma \vdash \lambda x : \alpha.\ e\ x \equiv e}$$

$$\frac{\Gamma \vdash p : \mathbb{P} \quad \Gamma \vdash h : p \quad \Gamma \vdash h' : p}{\Gamma \vdash h \equiv h'}$$

zf::empty	exists(s) { foreach(d) { !in(d, s) } }
zf::extent	foreach (a, b) { foreach (d) { in(d, a) <=> in(d, b) } => a = b }

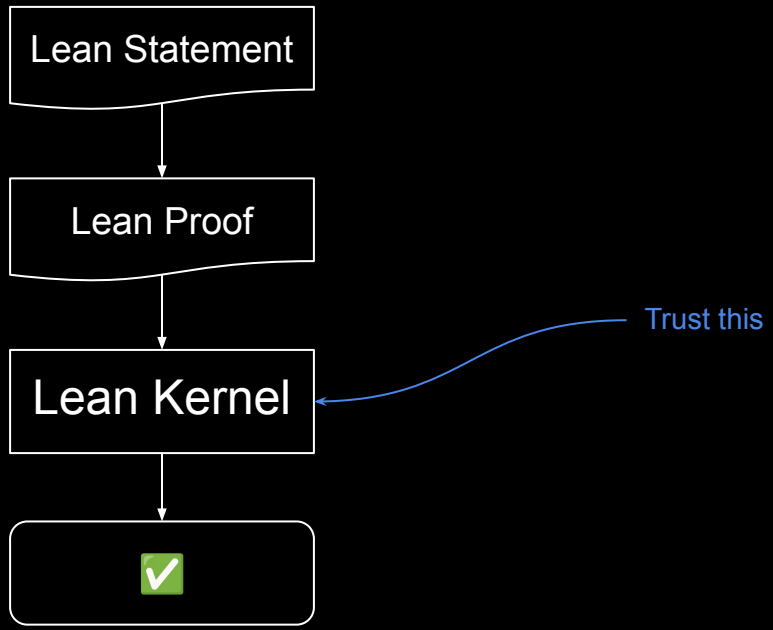
zf::inf	exists(s) {
	  in(zf::empty::s, s) &&
	  foreach(d) { in(d, s) => zf::single(d) as ds && zf::pair(d, ds) as p && zf::union(p) as u && in(u, s) }
	}

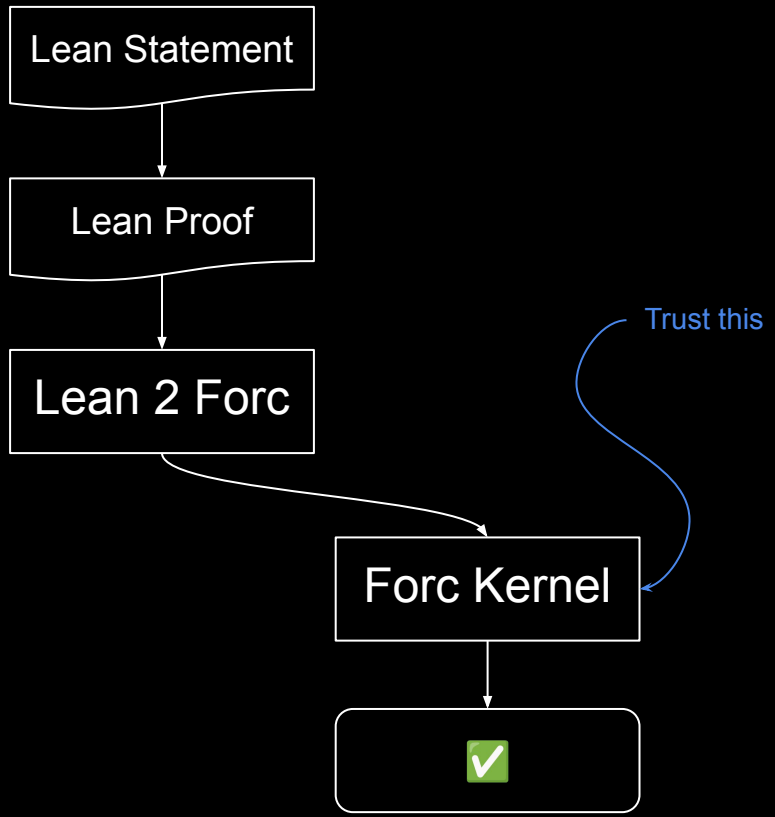
zf::power	foreach(a) { exists(s) { foreach(d) { in(d, s) <=> foreach(x) { in(x, d) => in(x, a) } } } }
zf::reg	foreach (x) { !(x = zf::empty::s) => exists (y) { in(y, x) && !exists (d) { in(d, x) && in(d, y) } } }

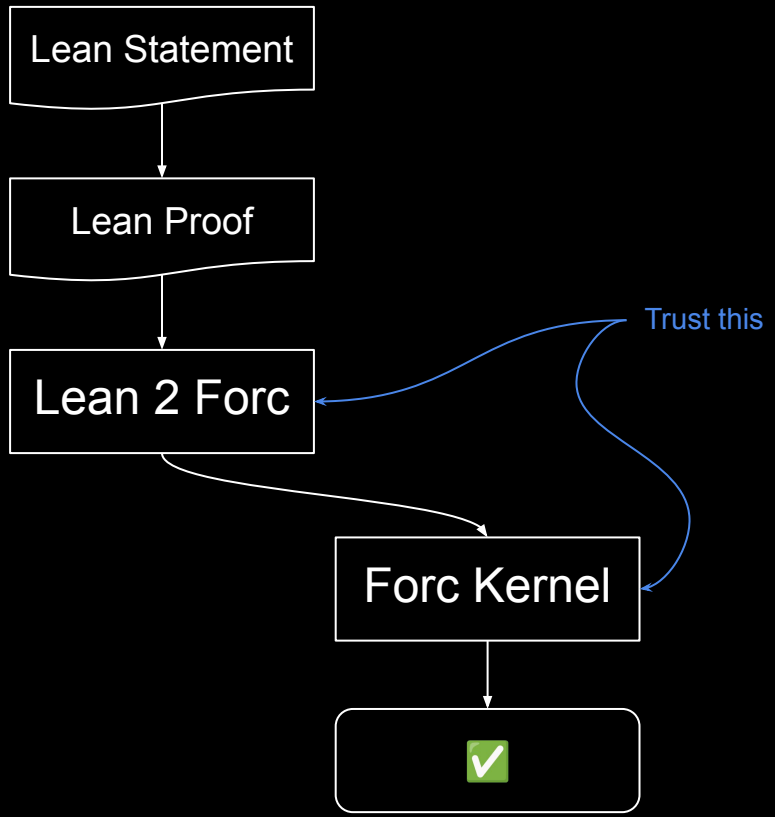
zf::replace	foreach (pred(p, q)) {
	  foreach (s) {
	  foreach (z) { in(z, s) => exists (y) { [pred](z, y) } && foreach (yp) { [pred](z, yp) => yp = y } } =>
	  exists (o) { foreach (x) { in(x, o) <=> exists (y1) { in(y1, s) && [pred](y1, x) } } }
	  }
	}

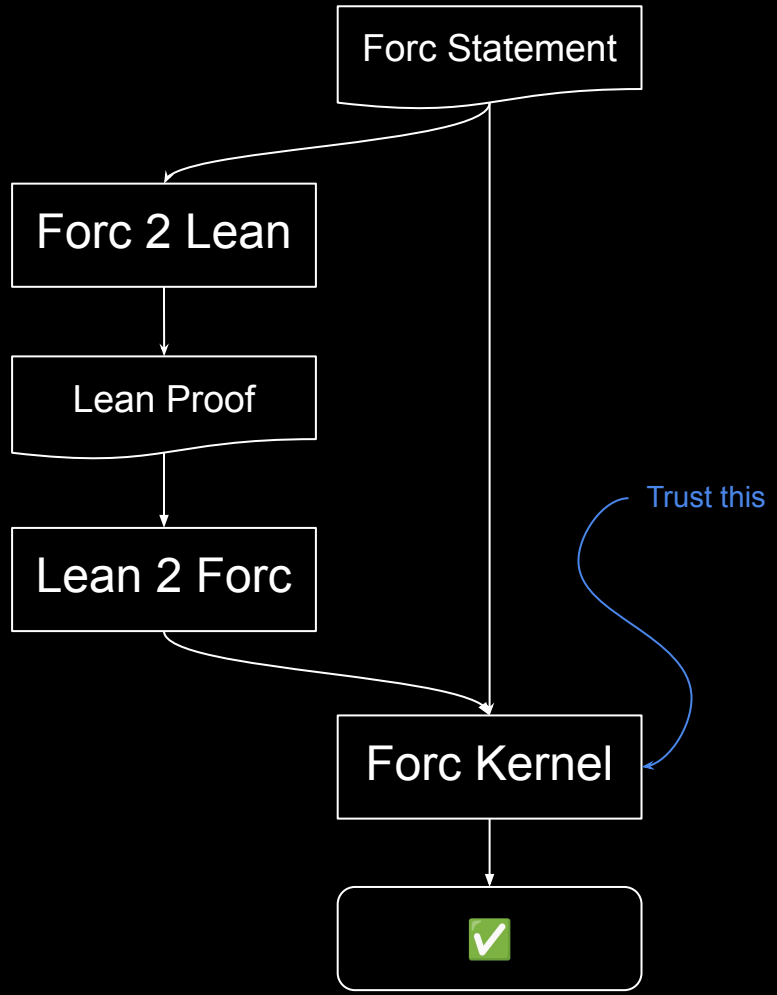
zf::union	foreach(s) { exists(u) { foreach(x) { in(x, u) <=> exists(c) { in(x, c) && in(c, s) } } } }

zf::inacc	`foreach (n) {`
	`  in(n, tp::nat::o) =>`
	`  exists (s) {`
	`  set::is_eqv(n, s) &&`
	`  foreach (k) { in(k, s) => @tp::card::is_inacc(k) }`
	`  }`
	`}`


Lean Statement
Lean Proof
Lean Kernel
✅
Trust this


Lean Statement
Lean Proof
Lean 2 Forc
Trust this
Forc Kernel
✅


Lean Statement
Lean Proof
Trust this
Lean 2 Forc
Forc Kernel
✅


Forc Statement
Forc 2 Lean
Lean Proof
Lean 2 Forc
Trust this
Forc Kernel
✅

Thank You

Get in touch

alex@near.ai

@AlexSkidanov

near